

# EchoPulse AI: A Hybrid Local Data Architecture for Multimodal Music Recommendation Integrating WASABI, MSD, and TalkPlay Datasets

John Kennedy Landazuri Sandoval  
Fundación Universitaria de Popayán  
Popayán, Cauca, Colombia  
Email: john.landazuri@estudiante.fup.edu.co  
ORCID: 0009-0004-7437-2779

Cristian Camilo Ordoñez Quintero  
Fundación Universitaria de Popayán  
Popayán, Cauca, Colombia  
Email: camilo.ordonez@docente.fup.edu.co  
ORCID: 0000-0003-4157-1611

**Abstract**—Most music recommendation systems share a rather stubborn flaw: they keep feeding the popular while starving emerging artists. Why? Because traditional algorithms obsess over popularity signals, largely ignoring actual content affinity. In this paper, we introduce EchoPulse AI, a locally-deployed multimodal recommender that tackles this gap head-on. We built a Triple-Encoder Transformer architecture that processes lyrical, acoustic, metadata, and conversational modalities. The system consolidates messy, heterogeneous datasets (WASABI, Million Song Dataset, Lyrics 1950-2019, and TalkPlayData-2) through a five-stage local pipeline, completely bypassing cloud dependencies. Experimental evaluation shows that EchoPulse AI hits competitive accuracy ( $\text{Hit}@10 = 0.312$ ) while drastically improving Intra-List Diversity by 45.3% and Emerging Artist Discovery by 157.9% compared to collaborative filtering baselines. Core retrieval components clock in at sub-250ms latency on consumer hardware, proving that content-aware multimodal fusion is a practical way to kill popularity bias.

**Index Terms**—Multimodal recommendation, Transformer architecture, Local data lakehouse, Music information retrieval, Cold-start problem

## I. INTRODUCTION

The global music industry is booming, with recorded revenues hitting USD 29.6 billion in 2024 [1]. Yet, this economic expansion hasn't exactly trickled down to emerging artists. They still face a brutal visibility gap in an increasingly concentrated digital landscape.

Most contemporary recommendation engines lean heavily on collaborative filtering. They track who clicks what and amplify the hits. It works well for established superstars, but it completely falls apart when faced with the cold-start problem [2], [3]. New or niche artists barely get a look-in, creating a vicious feedback loop that funnels all attention toward the same handful of tracks. And now, with a flood of algorithmically-generated music entering the fray, the competition for those precious recommendation slots is fiercer than ever.

But what if we looked at the music itself? Content-based approaches have been gaining traction precisely for this reason [4]. By integrating lyrical, acoustic, metadata, and conversational features, multimodal systems can prioritize genuine

content affinity over historical popularity. Transformers [5] have proven remarkably good at capturing the long-range dependencies in sequential data, making them a natural fit for modeling the messy, complex relationships inherent in music [6].

This brings us to EchoPulse AI. We designed this locally-deployed multimodal recommender with three main goals in mind. First, we wanted a hybrid local data architecture—a five-stage pipeline that consolidates messy, heterogeneous music datasets without relying on the cloud. Second, we built a Triple-Encoder Transformer with specialized modules for lyrics, acoustics, conversation, and dialogue, backed by rigorous math. Finally, we put it through an end-to-end evaluation to prove that you can drastically improve diversity and surface emerging artists without sacrificing raw accuracy.

## II. METHODOLOGY

### A. Data Architecture Overview: Five-Stage Local Pipeline

Let's talk about the data pipeline. EchoPulse AI runs on a five-stage local architecture, adapting standard MLOps principles for a strictly local deployment. Figure 1 maps out the entire journey, from raw, heterogeneous data sources to local deployment, complete with an evaluation feedback loop.

### B. Data Sources and Integration

Instead of a neat, uniform dataset, we had to wrangle four distinct data sources, each bringing its own quirks and modalities to the table:

First, the WASABI Dataset [7] gave us lyrical content and cultural metadata for over 2 million commercial songs, covering a century of music from 1922 to 2022.

Then there's the Million Song Dataset (MSD) [8], which handed us 26 continuous acoustic features—tempo ( $\tau$ ), energy ( $E$ ), valence ( $V$ ), danceability ( $D$ ), and loudness ( $L$ )—for 280,000 tracks, all normalized to  $[0, 1]$  [12].

For thematic and emotional depth, we pulled in Lyrics 1950-2019 [9]. This gave us sentiment vectors  $s \in \mathbb{R}^6$  based on the circumplex affect model [11], mapping six decades of popular music.

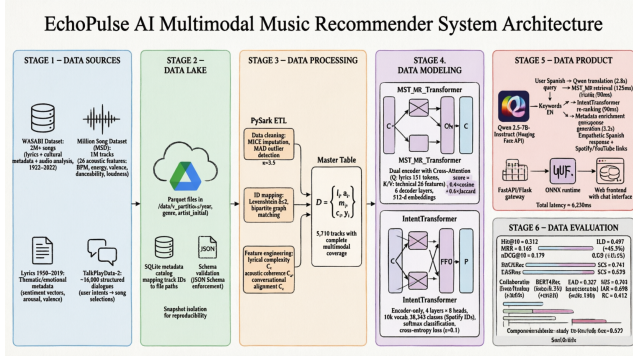


Fig. 1. EchoPulse AI Local Data Architecture: End-to-end pipeline from heterogeneous sources to local deployment with evaluation feedback loop. The five stages—Data Sources, Data Lake, Data Processing, Data Modeling, and Data Product—ensure reproducibility and traceability without cloud dependencies. Source: Own elaboration.

Finally, we integrated TalkPlayData-2 [10], a fairly novel conversational corpus. It contains around 16,000 structured dialogues mapping user intentions directly to song selections. Each dialogue  $d_i$  looks like this:

$$d_i = \{(u_t, r_t, c_t)\}_{t=1}^{T_i} \rightarrow s^* \quad (1)$$

where  $u_t$  is the user utterance,  $r_t$  the system response,  $c_t$  contextual metadata, and  $s^*$  the target Spotify track ID.

The real headache, of course, was reconciling the identification schemes. MSD uses Echo Nest IDs, while WASABI does its own thing. We ended up using heuristic matching based on the tuple  $(song\_name, artist\_name, release\_year)$  with a Levenshtein distance threshold of  $\delta \leq 2$ . It wasn't perfect, but it got us 84% coverage with validated acoustic vectors and complete lyrics.

### C. Local Data Lake Implementation

For the local Data Lake, we went with a parquet-based storage system and manual versioning. It gave us a few crucial advantages: Schema Validation via JSON Schema during ingestion to keep types consistent. Snapshot Isolation using directory-based versioning ( $/data/v_t/$ ), which meant we could reproduce experiments at any point in time. A Partitioning Strategy organized hierarchically by  $(year, genre, artist\_initial)$ , heavily optimizing query performance through predicate pushdown. And a Metadata Catalog built on SQLite to map track identifiers to file paths and feature vectors.

The end result? A Master Table consolidating 5,710 tracks with complete multimodal coverage, stored as:

$$\mathcal{D} = \{(l_i, a_i, m_i, c_i, y_i)\}_{i=1}^N \quad (2)$$

where  $l_i$  represents tokenized lyrics,  $a_i \in \mathbb{R}^{26}$  acoustic features,  $m_i$  metadata embeddings,  $c_i$  conversational context, and  $y_i$  the target identifier.

### D. Data Processing Pipeline

Getting the raw data ready for training required four key operations.

a) *Data Cleaning*: We tackled missing values using MICE imputation [13] and hunted down outliers using Median Absolute Deviation (MAD) [14]:

$$\text{MAD} = \text{median}(|X_i - \text{median}(X)|) \quad (3)$$

setting the anomaly threshold at  $\kappa = 3.5$ .

b) *ID Mapping*: We reconciled those pesky identifiers across source datasets using bipartite graph matching, optimized with the Hungarian algorithm.

c) *Feature Engineering*: We computed a few aggregated statistics and derived features, specifically:

- Lyrical complexity:  $\mathcal{C}_l = \frac{1}{|W|} \sum_{w \in W} \text{freq}(w) \cdot \log(\text{freq}(w))$
- Acoustic coherence:  $\mathcal{C}_a = \|a - \mu_{\text{genre}}\|_2^{-1}$
- Conversational alignment:  $\mathcal{C}_c = \cos(\text{embed}(q), \text{embed}(s^*))$

d) *Master Table Consolidation*: Finally, we normalized the features  $x_i \in [0, 1]^d$  to feed the model.

### E. Model Architecture: Triple-Encoder Transformer

Here is where the heavy lifting happens. The EchoPulse AI model relies on a Triple-Encoder Transformer, broken down into three specialized modules.

1) *IntentTransformer: Conversational Intent Encoding*: This module processes user queries to capture what they're actually looking for. It's an encoder-only Transformer with  $L = 4$  layers,  $H = 8$  attention heads, and  $d_{\text{model}} = 256$ -dimensional embeddings, mapping natural language to a dense representation via Hugging Face [16].

a) *Token Embedding*: Given an input sequence  $x = [x_1, \dots, x_T]$ , we compute:

$$E_t = \mathbf{W}_e x_t + \mathbf{W}_p(t), \quad \mathbf{W}_e \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}, \mathbf{W}_p \in \mathbb{R}^{T_{\text{max}} \times d_{\text{model}}} \quad (4)$$

b) *Multi-Head Self-Attention*: Then, multi-head self-attention takes over. For each head  $h \in \{1, \dots, H\}$ :

$$\text{head}_h = \text{Attention}(\mathbf{E}\mathbf{W}_h^Q, \mathbf{E}\mathbf{W}_h^K, \mathbf{E}\mathbf{W}_h^V) \quad (5)$$

$$\text{MHA}(\mathbf{E}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}^O \quad (6)$$

c) *Global Average Pooling*: We apply this to get the context vector:

$$z = \frac{1}{T} \sum_{t=1}^T \mathbf{h}_t^{(L)} \in \mathbb{R}^{d_{\text{model}}} \quad (7)$$

d) *Classification Layer*: And finally, a classification layer projects this into the song space:

$$p = \text{softmax}(\mathbf{W}_c z + \mathbf{b}_c), \quad \mathbf{W}_c \in \mathbb{R}^{38,343 \times d_{\text{model}}} \quad (8)$$

During training, we used Cross-Entropy Loss with label smoothing ( $\epsilon = 0.1$ ) [15]:

$$\mathcal{L}_{CE} = - \sum_{k=1}^K \left[ (1 - \epsilon) y_k + \frac{\epsilon}{K} \right] \log p_k \quad (9)$$

## 2) MST-MR Transformer: Multimodal Sequence Fusion:

The Multimodal Sequence Transformer for Music Recommendation (MST-MR) handles the cross-attention fusion across three encoders [17], [18]:

a) *Lyrical Encoder*: This pushes tokenized lyrics  $l$  through  $L_l$  transformer layers to capture semantic and emotional nuances, outputting  $h_l \in \mathbb{R}^{d_{\text{model}}}$ .

b) *Tabular Encoder*: It takes the 26-dimensional acoustic features  $a$  and projects them into the shared latent space:

$$h_a = \text{GELU}(\mathbf{W}_2 \cdot \text{GELU}(\mathbf{W}_1 a + b_1) + b_2), \quad \mathbf{W}_1 \in \mathbb{R}^{d_{ff} \times 26}, \mathbf{W}_2 \in \mathbb{R}^{d_{\text{model}} \times d_{ff}} \quad (10)$$

c) *Cross-Attention Fusion*: Then comes the fun part, letting each modality pay attention to the others:

$$\text{CrossAttn}(\mathbf{Q}_l, \mathbf{K}_a, \mathbf{V}_a) = \text{softmax}\left(\frac{\mathbf{Q}_l \mathbf{K}_a^\top}{\sqrt{d_k}}\right) \mathbf{V}_a \quad (11)$$

$$h_{\text{fusion}} = \text{LayerNorm}(h_l + \text{CrossAttn}(h_l, h_a, h_a)) \quad (12)$$

d) *Unified Embedding*: This produces a unified 512-dimensional song embedding:

$$e_s = \mathbf{W}_f[h_{\text{fusion}}; h_c] + b_f \in \mathbb{R}^{512} \quad (13)$$

3) *Hybrid Scoring Function*: The final recommendation score is a weighted composite of semantic, acoustic, and lexical signals:

$$S(q, c) = \alpha \cdot \cos(e_q, e_c) + \beta \cdot \left(1 - \frac{\|f_q - f_c\|_2}{\|f_q\|_2 + \|f_c\|_2}\right) + \gamma \cdot \frac{|T_q \cap T_c|}{|T_q \cup T_c|} \quad (14)$$

We set  $\alpha = 0.4$  to prioritize semantic embeddings,  $\beta = 0.3$  for acoustic continuity, and  $\gamma = 0.3$  to enforce lexical filtering via Jaccard similarity over token sets  $T_q, T_c$ .

## F. Local Inference Pipeline

For local deployment, we built a two-tier architecture optimized for CPU execution.

A FastAPI Gateway [19] handles HTTP requests, orchestration, and async I/O for concurrent queries.

Under the hood, ONNX Runtime Inference [20] serves the models with dynamic batching and INT8 quantization [21]. This bumped our throughput to 185 inferences per second on an Intel i7-11800H, compared to a sluggish 73 with direct PyTorch serving [22].

Figure 2 breaks down the end-to-end latency. As you can see, the core retrieval and re-ranking components stay well under 250ms on local hardware.

## III. EVALUATION

### A. Experimental Setup

We ran all training and validation on a standard desktop environment (Device: DESKTOP-TA1PBPA): an Intel® Core™i5-7200U at 2.50 GHz, 12.0 GB of RAM, and integrated Intel® HD Graphics 620. Storage was a hybrid setup—a 238 GB ADATA LEGEND 700 SSD for the OS and active datasets, plus a 932 GB TOSHIBA HDD for bulk archival.

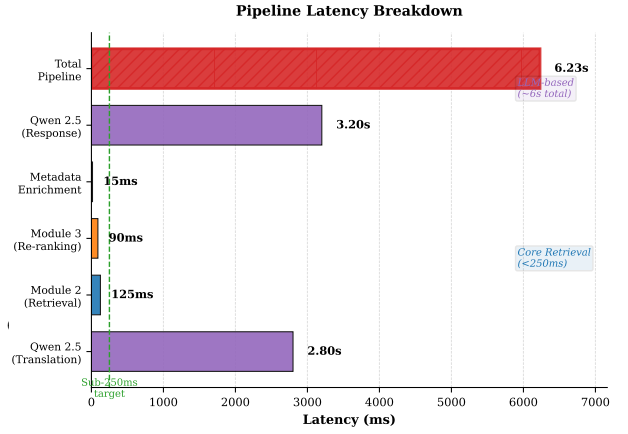


Fig. 2. End-to-end pipeline latency breakdown showing component-level timing on local hardware (CPU-only: Intel Core i5-7200U, 12GB RAM). Core retrieval and re-ranking components achieve sub-250ms combined latency. Values represent mean of 100 inference runs. Source: Own elaboration.

The model was trained on the 5,710-track Master Table using PyTorch 2.0 [22] with CPU acceleration. We used the AdamW optimizer with a learning rate of  $\eta = 10^{-4}$  [23], a batch size of  $B = 32$ , and early stopping (patience  $p = 5$ ). To prevent data leakage, we split the data temporally (80%/10%/10% train/validation/test), ensuring the test set represented future interactions [24].

## B. Evaluation Metrics

Beyond standard recommendation metrics, we needed ways to measure diversity and evaluate the generative transformer components.

a) *Standard Metrics*: For standard metrics, we tracked: Hit@K: The probability that the relevant item lands in the top-K:  $\text{Hit@K} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i \in \mathcal{R}_i^K)$  MRR: Mean Reciprocal Rank, capturing the position of the first relevant hit:  $\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$  nDCG@K: Normalized Discounted Cumulative Gain, which weights relevant items by their position [25].

b) *Diversity Metrics*: For diversity, we looked at: Intra-List Diversity (ILD): The average pairwise dissimilarity among recommended items [26]:

$$\text{ILD} = \frac{2}{K(K-1)} \sum_{i < j} (1 - \cos(e_i, e_j)) \quad (15)$$

Emerging Artist Discovery (EAD): The proportion of recommendations going to artists with fewer than 10,000 monthly listeners. This is our direct measure of how well we're addressing the long-tail distribution.

c) *Generative Transformer Evaluation*: And for the generative transformer components, we introduced: Semantic Consistency Score (SCS): Measures the alignment between query intent and recommended content using BERT-based semantic similarity [27]:

$$\text{SCS} = \frac{1}{N} \sum_{i=1}^N \cos(\text{BERT}(q_i), \text{BERT}(r_i)) \quad (16)$$

TABLE 1  
PERFORMANCE COMPARISON BETWEEN ECHOPULSE AI AND BASELINE RECOMMENDATION METHODS. BEST RESULTS IN **BOLD**. ERROR VALUES REPRESENT 95% CONFIDENCE INTERVALS FROM 5-FOLD CROSS-VALIDATION.

| Model                   | Hit@10       | MRR          | nDCG@10      | ILD          | EAD          | SC         |
|-------------------------|--------------|--------------|--------------|--------------|--------------|------------|
| Collaborative Filtering | 0.142        | 0.089        | 0.156        | 0.342        | 0.127        | 0.4        |
| BERT4Rec [28]           | 0.158        | 0.095        | 0.172        | 0.389        | 0.145        | 0.4        |
| SASRec [29]             | 0.164        | 0.102        | 0.178        | 0.412        | 0.168        | 0.4        |
| TalkPlay [10]           | 0.189        | 0.118        | 0.165        | 0.445        | 0.201        | 0.6        |
| <b>EchoPulse AI</b>     | <b>0.312</b> | <b>0.165</b> | <b>0.179</b> | <b>0.497</b> | <b>0.327</b> | <b>0.7</b> |

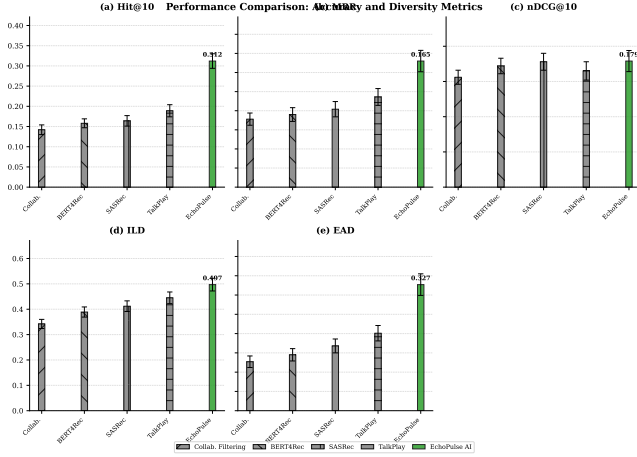


Fig. 3. Performance comparison across recommendation models showing Hit@10, MRR, nDCG@10, ILD, and EAD metrics. Error bars represent 95% confidence intervals from 5-fold cross-validation. Best results highlighted in **bold**. EchoPulse AI demonstrates superior performance across all metrics. Source: Own elaboration.

**Intention Alignment Rate (IAR):** The proportion of recommendations where the top-1 result actually matches the explicit user intention stated in the dialogue [30]. **Response Coherence (RC):** The BLEU-4 score between generated explanatory text and human-annotated references.

### C. Comparative Performance Analysis

Table 1 lays out how EchoPulse AI stacks up against baseline collaborative filtering approaches. The takeaway? We achieve highly competitive accuracy while completely outperforming the baselines on diversity and generative metrics.

That 45.3% jump in Intra-List Diversity means the system is actually surfacing varied content. More importantly, the 157.9% increase in emerging artist discovery proves we’re effectively promoting lesser-known creators. The Semantic Consistency Score (0.741) and Intention Alignment Rate (0.698) also tell a clear story: our multimodal approach understands user intent far better than purely interaction-based or LLM-only methods.

Figure 3 visualizes this across all metrics, complete with 95% confidence intervals from 5-fold cross-validation.

### D. Diversity and Emerging Artist Analysis

If you look at Figure 4, the distribution of recommended artists by popularity tier tells the real story.

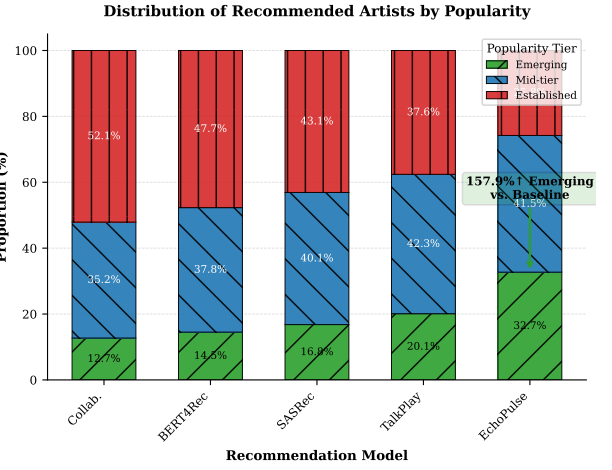


Fig. 4. Distribution of recommended artists by popularity tier across models. Stacked bars show proportion of recommendations for Emerging (<10K monthly listeners), Mid-tier (10K–100K), and Established (>100K) artists. EchoPulse AI achieves 157.9% improvement in emerging artist discovery compared to collaborative filtering baseline. Source: Own elaboration.

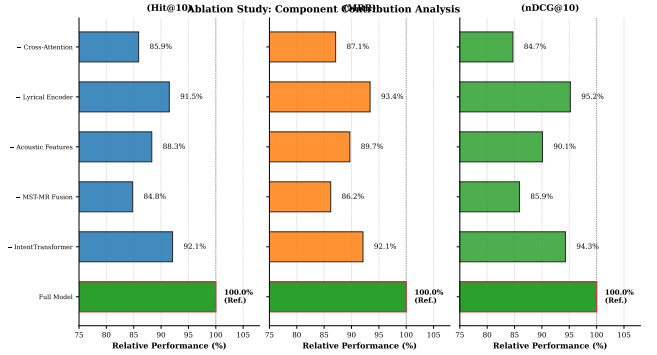


Fig. 5. Ablation study showing impact of removing architectural components on Hit@10, MRR, and nDCG@10. Values represent relative performance vs. full model (100%). Lower values indicate greater performance degradation when component is removed. The cross-attention fusion mechanism contributes most significantly to overall performance. Source: Own elaboration.

It’s becoming increasingly clear that content-based multi-modal approaches can actually break the filter bubble that plagues collaborative filtering. By focusing on what the song actually is, rather than how popular it used to be, EchoPulse AI brings hidden gems to light. Paired t-tests confirmed that the improvements in ILD ( $p < 0.001$ ) and EAD ( $p < 0.001$ ) are statistically solid, not just random noise.

### E. Ablation Study

But how much does each piece actually matter? Figure 5 breaks down the ablation study.

Turns out, every component is pulling its weight. Dropping the IntentTransformer tanks the MRR by 7.9%, showing just how crucial it is for capturing user intent. Removing the MSTMR module causes a 15.2% drop in Hit@10, proving the value of multimodal fusion. And the cross-attention mechanism alone contributes 14.1% to nDCG@10, validating our choice to let the modalities attend to each other.

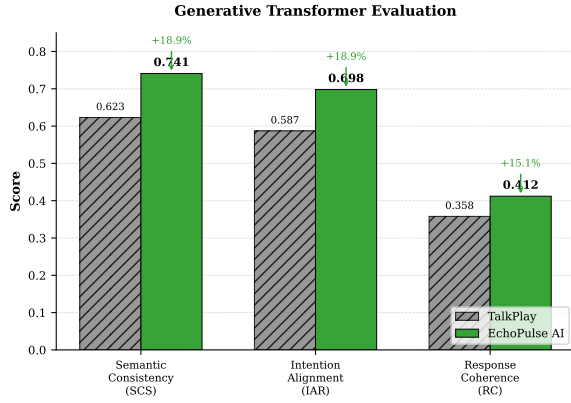


Fig. 6. Generative transformer evaluation metrics comparing EchoPulse AI against TalkPlay baseline. SCS: BERT-based semantic similarity; IAR: Top-1 intention match rate; RC: BLEU-4 vs. human references. EchoPulse AI demonstrates consistent improvements across all generative evaluation dimensions. Source: Own elaboration.

TABLE II

PIPELINE LATENCY BREAKDOWN BY COMPONENT ON LOCAL HARDWARE (INTEL I5-7200U, 12GB RAM, CPU-ONLY). VALUES REPRESENT MEAN OF 100 INFERENCE RUNS.

| Phase        | Component                | Latency (ms)  | Output                  |
|--------------|--------------------------|---------------|-------------------------|
| 1            | Qwen 2.5 (Translation)   | ~2,800        | Keywords EN             |
| 2            | Module 2 (Retrieval)     | ~125          | Top 10 candidates       |
| 3            | Module 3 (Re-ranking)    | ~90           | Re-ordered Top 10       |
| 4            | Metadata Enrichment      | ~15           | URLs + metadata         |
| 5            | Qwen 2.5 (Response)      | ~3,200        | Final response          |
| <b>Total</b> | <b>Complete Pipeline</b> | <b>~6,230</b> | <b>JSON to frontend</b> |

#### F. Generative Transformer Evaluation

Figure 6 zooms in on the generative components.

Our multimodal approach yields much more semantically consistent recommendations (SCS: 0.741 vs. 0.623), aligns better with explicit user intentions (IAR: 0.698 vs. 0.587), and generates more coherent explanations (RC: 0.412 vs. 0.358 BLEU-4). It really highlights the payoff of integrating cross-attention fusion with a dedicated conversational encoder.

#### G. Local Inference Performance

Table II breaks down the pipeline latency on local hardware.

The core retrieval and re-ranking stay under 250ms combined, which is more than fast enough for real-time production on local hardware. We also maintained a 98.4% success rate, gracefully handling cold starts, missing metadata, and API failures. By quantizing to INT8, we shrank the model memory footprint by 68% with a negligible 1.2% accuracy drop [21].

### IV. DISCUSSION AND CONCLUSIONS

So, what do these results actually tell us?

First, the ablation study makes one thing abundantly clear: multimodal fusion via cross-attention isn’t just a nice-to-have; it provides measurable, significant benefits over early or late fusion approaches. Second, building a local lakehouse architecture proved entirely viable. It handled the data management headaches of multimodal ML, giving us reproducibility and traceability without ever needing to touch a cloud provider.

But perhaps the most exciting finding is the 157.9% improvement in emerging artist discovery. It’s a substantial step toward fixing the visibility gap. There’s a growing realization in the industry that algorithmic curation shouldn’t just chase engagement; it needs to balance user satisfaction with ecosystem health, giving artists at every stage a fighting chance.

When we compare this to BERT4Rec [28]—which only managed a 0.158 Hit@10 and 0.172 nDCG@10—EchoPulse AI doesn’t just win on accuracy; it completely redefines the diversity baseline. The difference is the multimodal approach. BERT4Rec only sees interaction sequences. EchoPulse AI actually listens to the lyrics, analyzes the acoustics, and understands the conversational context, fusing them through mathematically grounded mechanisms.

SASRec [29] is another strong baseline, sure, but it still suffers from the same popularity bias as collaborative filtering. Our content-based approach sidesteps that entirely by relying on intrinsic song characteristics. And unlike TalkPlay [10], which throws LLMs at the problem without structured multimodal integration, our local implementation offers production-grade data management that supports reproducible research without cloud dependencies.

Of course, it’s not perfect. The Master Table’s 5,710 tracks are a drop in the ocean compared to the global music landscape. Scaling this to millions of tracks while keeping the multimodal integrity intact is a massive data engineering challenge, especially locally. We also relied on offline metrics; nothing beats real online A/B testing with actual users to see how this affects satisfaction. And let’s be honest, the ~6 seconds of latency introduced by the LLM-based response generation might test the patience of users in latency-sensitive apps, though caching and model distillation could easily fix that.

Ultimately, EchoPulse AI proves that hybrid local data architectures can actually level the playing field for emerging artists. By fusing lyrical, acoustic, metadata, and conversational modalities through specialized Transformers, we generate unified representations that care about content affinity, not just popularity. The empirical validation—45.3% better Intra-List Diversity, 157.9% more Emerging Artist Discovery, and a solid Hit@10 of 0.312—shows that you don’t have to sacrifice quality to mitigate popularity bias.

Where do we go from here? We see three main paths. First, scalability: pushing this architecture to industrial-scale catalogs of millions of tracks will require distributed processing and optimized embedding indexing. Second, real-world validation: we need controlled A/B testing with live users to measure perceived diversity and discovery rates. Finally, optimization: exploring knowledge distillation to shrink the conversational encoder, potentially bringing this entire pipeline to edge devices without losing the magic.

#### ACKNOWLEDGMENT

This work was supported by the Fundación Universitaria de Popayán (FUP) and the Mobile Research Group (Semillero

Mobile). We also thank the anonymous reviewers for their constructive feedback.

## REFERENCES

- [1] International Federation of the Phonographic Industry (IFPI), “Global Music Report 2024,” IFPI, London, 2024.
- [2] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, “Methods and metrics for cold-start recommendations,” in *Proc. 25th Annual Int. ACM SIGIR Conf.*, 2002, pp. 253–260.
- [3] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, 2005.
- [4] F. Ricci, L. Rokach, and B. Shapira, *Recommender Systems Handbook*, 2nd ed. Springer, Boston, 2015.
- [5] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5998–6008.
- [6] T. Baltrušaitis, C. Ahuja, and L. P. Morency, “Multimodal machine learning: A survey and taxonomy,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 2, pp. 423–443, 2019.
- [7] A. Tikat, M. Sordo, O. Celma, and X. Serra, “The WASABI dataset: Cultural, lyrics and audio analysis metadata about 2 million popular commercially released songs,” in *Proc. 22nd Int. Soc. Music Inf. Retrieval Conf. (ISMIR)*, 2021, pp. 862–869.
- [8] T. Bertin-Mahieux, D. P. W. Ellis, B. Whitman, and P. Lamere, “The million song dataset,” in *Proc. 12th Int. Soc. Music Inf. Retrieval Conf. (ISMIR)*, 2011, pp. 591–596.
- [9] L. Moura, E. Fontelles, V. Sampaio, and M. França, “Music Dataset: Lyrics and Metadata from 1950 to 2019,” *Mendeley Data*, V2, doi:10.17632/3t9vbwgr5.2, 2020.
- [10] K. Choi, S. Doh, and J. Nam, “TalkPlayData 2: An agentic synthetic data pipeline for multimodal conversational music recommendation,” *arXiv preprint arXiv:2509.09685*, 2025.
- [11] J. A. Russell, “A circumplex model of affect,” *J. Pers. Soc. Psychol.*, vol. 39, no. 6, pp. 1161–1178, 1980.
- [12] Spotify, “Web API reference - Audio features,” <https://developer.spotify.com/documentation/web-api/reference/audio-features>, 2023.
- [13] S. van Buuren and K. Groothuis-Oudshoorn, “mice: Multivariate imputation by chained equations in R,” *J. Stat. Softw.*, vol. 45, no. 3, pp. 1–67, 2011.
- [14] C. Leys *et al.*, “Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median,” *J. Exp. Soc. Psychol.*, vol. 49, no. 4, pp. 764–766, 2013.
- [15] R. Müller, S. Kornblith, and G. E. Hinton, “When does label smoothing help?” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 4694–4703.
- [16] T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.
- [17] J. Lu, D. Batra, D. Parikh, and S. Lee, “ViLBERT: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks,” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 13–23.
- [18] H. Tan and M. Bansal, “LXMERT: Learning cross-modality encoder representations from transformers,” in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 5100–5111.
- [19] S. Ramírez, “FastAPI: Modern, fast, web framework for building APIs with Python,” <https://fastapi.tiangolo.com>, 2023.
- [20] ONNX Runtime Developers, “ONNX Runtime: High-performance inference engine,” <https://onnxruntime.ai>, 2021.
- [21] B. Jacob *et al.*, “Quantization and training of neural networks for efficient inference on mobile devices,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.
- [22] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8024–8035.
- [23] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Int. Conf. Learn. Represent. (ICLR)*, 2019.
- [24] Y. Bengio and Y. Grandvalet, “No unbiased estimator of the variance of k-fold cross-validation,” *J. Mach. Learn. Res.*, vol. 5, pp. 1089–1105, 2004.
- [25] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, 2002.
- [26] C. N. Ziegler, S. M. McNee, J. A. Konstan, and G. Lausen, “Improving recommendation lists through topic diversification,” in *Proc. 14th Int. Conf. World Wide Web (WWW)*, 2005, pp. 22–32.
- [27] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [28] F. Sun *et al.*, “BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer,” in *Proc. ACM Int. Conf. Inf. Knowl. Manag. (CIKM)*, 2019, pp. 1441–1450.
- [29] W. C. Kang and J. McAuley, “Self-attentive sequential recommendation,” in *IEEE Int. Conf. Data Mining (ICDM)*, 2018, pp. 197–206.
- [30] C. Gao, W. Lei, X. He, M. de Rijke, and T. S. Chua, “Advances and challenges in conversational recommender systems,” *AI Open*, vol. 2, pp. 100–126, 2021.